

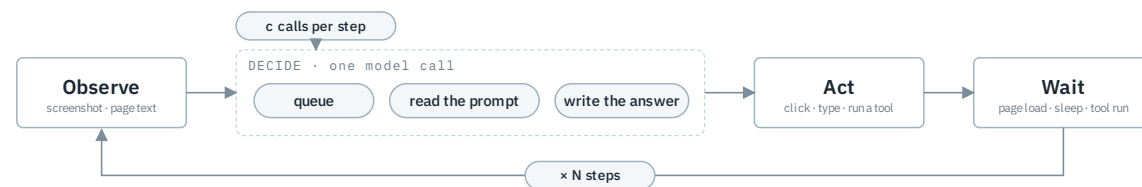
Where the time and the money go

Why LLM agents that operate software are slow and expensive, and how the two are linked.

Every number comes from a published measurement of a computer-use, web or coding agent, or from a vendor's own price page (2024–2026); the conditions of each measurement are stated next to it, and the full references are at the end of the part.

September 2026

The loop every agent runs — Part 1 puts a number on each term

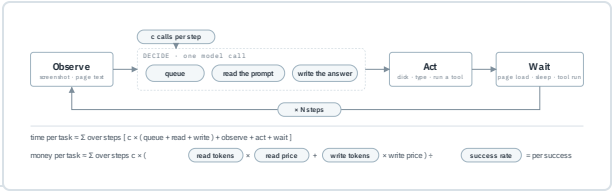


time per task $\approx \sum \text{over steps} [c \times (\text{queue} + \text{read} + \text{write}) + \text{observe} + \text{act} + \text{wait}]$

money per task $\approx \sum \text{over steps} c \times (\text{read tokens} \times \text{read price} + \text{write tokens} \times \text{write price}) \div \text{success rate} = \text{per success}$

read: the prompt is processed all at once · write: the answer comes out one token at a time · N passes, c model calls per pass

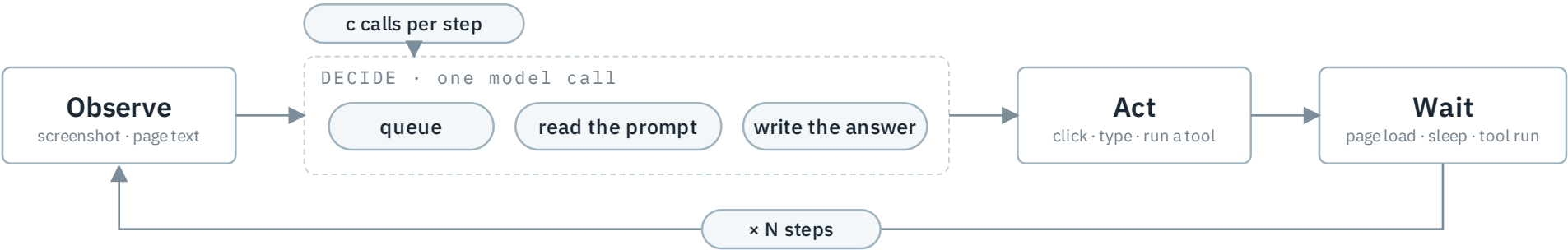
The loop, with every term on it



here: the loop and its two equations → to: why the reading term grows fastest

One task = one loop run N times. Each pass observes the screen or page, makes at least one model call, acts, and waits. A call has three segments: **queueing** at the provider; **reading** the prompt — all tokens processed together, cheap per token; **writing** the answer — one token at a time, about 12 ms each (81.7 tokens per second measured for Claude Opus 5.5, Artificial Analysis, 2026) and 5× the read price.

Observation, action, waiting and queueing enter only the time equation; read and write tokens enter both. That asymmetry is the whole structure behind “how slow and expensive are linked”.

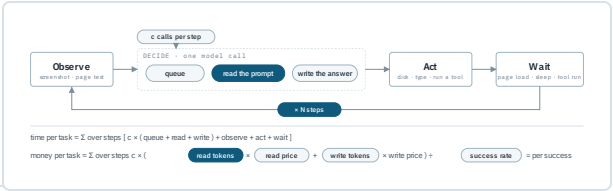


time per task ≈ Σ over steps [c × (queue + read + write) + observe + act + wait]

money per task ≈ Σ over steps c × (read tokens × read price + write tokens × write price) ÷ success rate = per success

N	passes — task length, action granularity, idling	c	model calls per pass — the harness (the program around the model): one call, or plan + judge + reflect
read	tokens read per call — history length, screenshot size	write	tokens written per call — thinking mode, model size
price	model tier, fast mode, cache (a cached read costs 0.1×)	observe · act · wait	environment time — browser, desktop sandbox, fixed sleeps

Reading grows with the square of the steps; writing only linearly

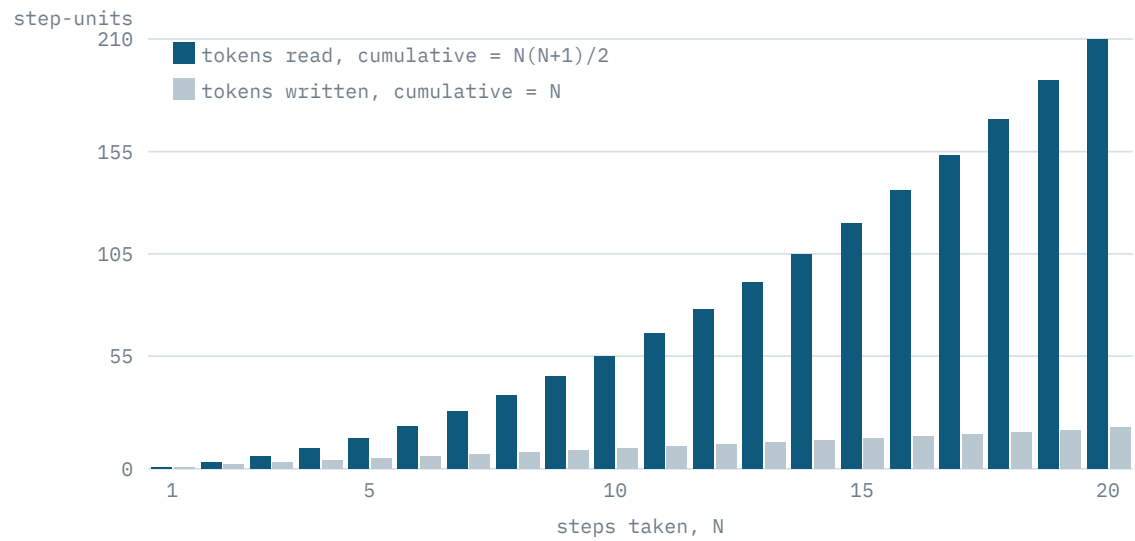


from the loop → here: the shape of the reading term → to: the five sources of slowness

Writing per step is roughly constant, so N steps write about $N \times W$ tokens — linear.

Reading at step k re-sends the $k-1$ earlier screenshots and messages — one screenshot is 1,000–1,800 tokens (Anthropic, 2026a) — so the sum over a task is about $N^2/2$ times the tokens added per step: 10 steps read 55 step-units of history, 20 steps read 210 (calc.). Doubling the steps roughly quadruples the reading and only doubles the writing.

This assumes the full history is re-sent with no cache. Trimming history flattens the curve; prompt caching cuts the read price to $0.1\times$ but leaves the shape.



Tokens read and written over a task, in units of “tokens added per step”, if every step re-sends the whole history (calc.).

MEASURED ON REAL RUNS

Cost “grows quadratically with the number of steps”

Step-by-step accounting of two multi-call harnesses (GTA1, Agent S2) on 39 OSWorld desktop tasks; the later steps are up to $3\times$ slower than the early ones because the prompt at step k carries the $k-1$ earlier screenshots — “dominated by prefill”, the reading segment.

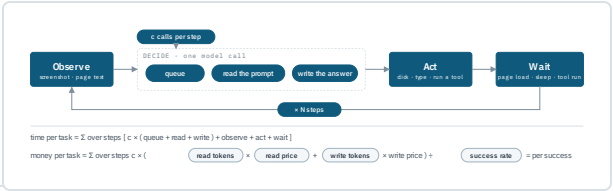
(Abhyankar, Qi & Zhang, 2026)

WHAT IT MEANS FOR ACCELERATION

The cheapest token is the one never re-read

Any method that shortens the history, shrinks each observation, or avoids a pass altogether attacks the fastest-growing term; a faster model only scales the per-token constant.

Slowness has five sources

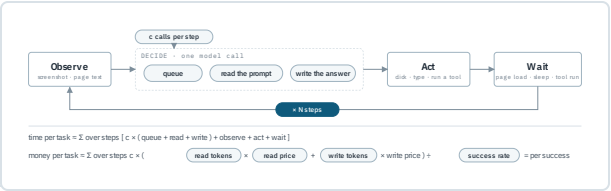


from the two equations → here: every cause of slowness, in five classes → to: one class per page

Four of the five are terms of the time equation; the fifth is the structure that adds them up. Which class dominates depends on the harness, not on the kind of agent (page 10). Every cause on the next pages has an entry in Appendix A1 with its measurement conditions.

time per task = $\frac{\sum \text{over N steps}}{I \cdot \text{STEPS}}$	$c \times \frac{1}{II \cdot \text{CALLS PER STEP}}$	$\frac{(\text{queue} + \text{read} + \text{write})}{III \cdot \text{EACH CALL}}$	$\frac{+ \text{observe} + \text{act} + \text{wait}}{IV \cdot \text{ENVIRONMENT}}$	$\frac{\text{all added, never overlapped}}{V \cdot \text{SERIAL}}$
I · TOO MANY STEPS (N) 318 tool calls per task Hour-scale tasks take hundreds of passes: 318 tool calls per task on OSWorld 2.0 with one action per step (Claude Opus 4.7, 108 tasks); in a median text-agent task two thirds of the steps are pure navigation. (XLANG Lab, 2026; Wong et al., 2026)	II · TOO MANY CALLS PER STEP (C) 4–12 planning calls per judging call Harnesses that plan, judge and reflect make several calls per pass; sampling candidates multiplies them again; a coding agent makes 6.6 model calls per user turn. (Abhyankar, Qi & Zhang, 2026; Liu et al., 2026)	III · EACH CALL IS SLOW Queue, read, write, think Identical requests vary up to 69× in latency by time of day; reading gets slower every step (up to 3×); writing one token at a time takes 91–98.6% of model time once the cache is warm; thinking modes multiply the writing. (Bian et al., 2025; Abhyankar, Qi & Zhang, 2026; Yuan et al., 2026)	IV · THE ENVIRONMENT IS SLOW 3–26 s per observation · 6.6 s per browser action Building an accessibility tree, executing in the browser, sleeping a fixed 2–3 s after every action, and tool runs with a long tail: 4% of tool calls take 85% of tool time. (Abhyankar, Qi & Zhang, 2026; Wong et al., 2026; Zhu et al., 2026)	V · EVERYTHING IS SERIAL concurrency within a turn: 1.15 Observe, decide, act and wait run strictly one after another, so task time is the sum of the four classes above; fixing one term saves only its own share — which is why the shares (page 10) must be known before anything is accelerated. (Liu et al., 2026)

I · Too many steps



from the five classes → here: why the number of passes N gets large → to: calls per step

N multiplies everything. It gets large in five ways: the task is long; each pass does a single action; navigation-only passes still go through the model; the agent idles or loops; it acts on a stale screen and has to redo the work.

I-1, I-2 · LONG TASKS, ONE ACTION PER PASS

318 tool calls per task, mean

OSWorld 2.0, 108 long desktop tasks, Claude Opus 4.7 with maximum thinking, one action per step, 500-step cap. Letting one call issue several actions cuts the same model to 160.7 steps and Claude Opus 4.8 to 103 steps (481.8 calls).

(XLANG Lab, 2026)

I-3 · NAVIGATION-ONLY PASSES

66.7% of steps are pure navigation

Median task across 151 WebVoyager tasks on live websites, three text agents (Browser-Use, AgentOccam, WebVoyager) driven by GPT-4o: paging, scrolling and opening the next page each cost a full model call.

(Wong et al., 2026)

I-4 · IDLING AND DEAD LOOPS

66% of steps wasted; one loop = 27 min, \$8.47

GTA1 harness (o3 plans and judges, GTA1-7B locates elements) on 39 OSWorld desktop tasks: in failed tasks that ran past 50 steps, 66% of steps were repeats; one element-locating loop repeated the same step 18 times — 27 minutes and \$8.47 at list price without caching.

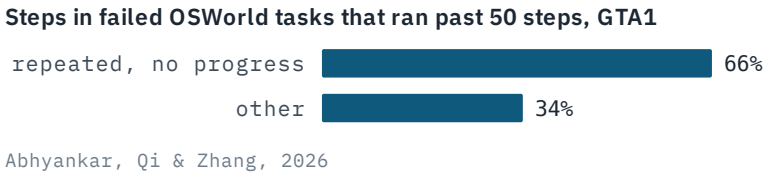
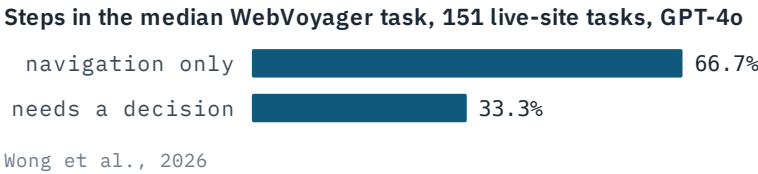
(Abhyankar, Qi & Zhang, 2026)

I-5 · ACTING ON A STALE SCREEN

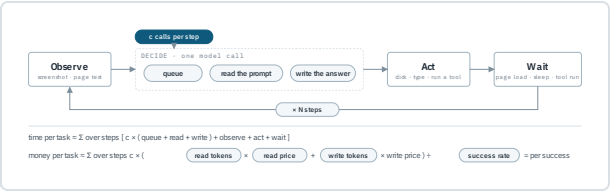
a recorded failure mode, share not reported

A screenshot taken before the page has changed leads to a click on the old layout and a repair afterwards; OSWorld 2.0 records “stale interface state” as one of its failure modes but gives no share.

(XLANG Lab, 2026)



II · Too many calls per step



from the number of passes → here: model calls per pass, c → to: what happens inside one call

c multiplies N. A harness that plans, judges and reflects makes several calls per pass; sampling several candidate actions per step multiplies the calls again; coding agents call the model many times inside one user turn.

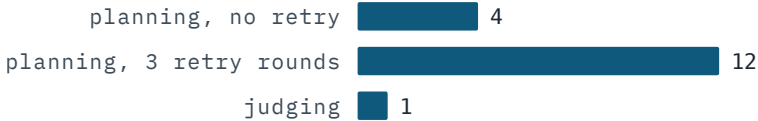
II-1 · PLAN + JUDGE + REFLECT

4–12 planning calls per judging call

GTA1 runs four parallel planners per step, retries up to three rounds, then one judging call picks; Agent S2 adds a reflection call per step, so planning takes 53% and reflection 34% of task time. Both measured step by step on 39 OSWorld desktop tasks.

(Abhyankar, Qi & Zhang, 2026)

Model calls per step in the GTA1 harness, per judging call



Abhyankar, Qi & Zhang, 2026 · 4 parallel planners, up to 3 rounds

II-2 · SAMPLING CANDIDATES

1 → 10 candidates: 38.8% → 43.2% success for 9.6× the tokens

Same model and harness (gpt-oss-120b, one-call-per-step ReAct) on 165 WebArena-Lite tasks; tokens per task rise from 96K to 920K, i.e. 9.6× the tokens for 4.4 points of accuracy.

(Lee et al., 2026)

Tokens per task vs candidates per step, WebArena-Lite, gpt-oss-120b



Lee et al., 2026 · 165 tasks

II-3 · CODING AGENTS

6.6 model calls per user turn

GitHub Copilot coding agent, production telemetry for the first week of June 2026, 13.5 million sessions: the model is called back and forth several times before the turn ends.

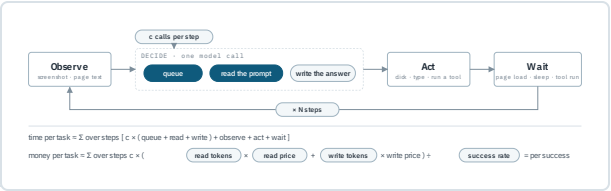
(Liu et al., 2026)

Model calls per user turn, GitHub Copilot coding agent



Liu et al., 2026 · production telemetry, June 2026

III · Each call is slow: queueing and reading



from calls per step → here: the first two segments of a call → to: writing, thinking, model size

Two segments before a single token is produced. Queueing at the provider costs nothing but counts as model time. Reading re-processes the entire history on every call, so it grows with the step number, with the size of each observation, and with whatever the cache failed to keep.

III-1 · QUEUEING

up to 69× latency spread for the same request

Client-side measurement of 15 models across 5 providers: requests of equal length, sent at different times of day, differ in latency by up to 69×. Not billed; usually reported as “model time”.

(Bian et al., 2025)

Latency of identical requests by time of day, 15 models, 5 providers



Bian et al., 2025 · measured from the client

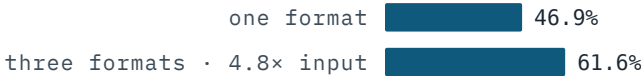
III-2 · READING GROWS WITH THE STEP

later steps up to 3× slower · 4.8× input when the observation is rich

OSWorld-Human: in the multi-call harnesses the prompt at step k carries the k−1 earlier screenshots, and the authors attribute the slowdown to the reading segment (“dominated by prefill”). AgentSysBench: switching one WebArena agent (Kimi-K2.6) from a single observation format to accessibility tree (the text list of on-screen elements) + HTML + screenshot multiplied the input 4.8× and raised the model’s share of time from 46.9% to 61.6%.

(Abhyankar, Qi & Zhang, 2026; Chang et al., 2026)

Model share of step time as the observation grows, WebArena, Kimi-K2.6



Chang et al., 2026 · AgentSysBench · accessibility tree + HTML + screenshot

III-2 · WHEN THE CACHE IS EVICTED

request latency up to 7.14× under load

SWE-Agent and OpenHands running GLM-4.6 on 8 H100s for SWE-bench Lite: as concurrency rises the cache of already-read prompt (the KV cache) is evicted and re-read, and request latency rises up to 7.14×.

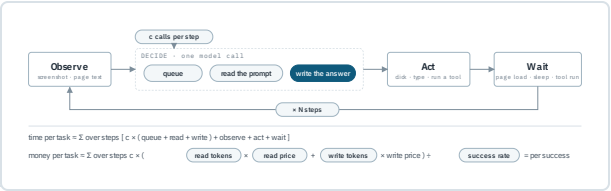
(Kang et al., 2026)

Request latency when the cache is evicted, GLM-4.6 on 8 H100s



Kang et al., 2026 · SWE-bench Lite

III · Each call is slow: writing, thinking, model size



from queueing and reading → here: the writing segment → to: the environment

Writing is one token at a time, so it is the slowest segment per token and the most expensive. Thinking modes add tokens that are written before the answer; larger models take longer per token. Neither reliably buys accuracy.

III-3 · WRITING DOMINATES A WARM CALL

91–98.6% of model time

One-call-per-step ReAct agents on five benchmarks, Qwen3.6-27B and Gemma4-31B served locally (vLLM serving engine, 2 H100s): with the cache warm, token-by-token generation takes 91–98.6% of the model’s time. Desktop agent UFO2 (GPT-4o / o1 API): about 10 s per model call, the largest item in every configuration.

(Yuan et al., 2026; Zhang et al., 2025)

Output tokens per task on the same 108 OSWorld 2.0 tasks



XLANG Lab, 2026

III-4 · THINKING MODES

224K vs 37K output tokens per task

Same 108 OSWorld 2.0 tasks: Claude Opus 4.8 writes 224,000 output tokens per task, GPT-5.5 writes 37,000; the writing is billed as output and, on Claude Opus 5.5, cannot be switched off.

(XLANG Lab, 2026; Anthropic, 2026b)

Share of model time spent writing, warm cache, local 27–31B models



Yuan et al., 2026 · Qwen3.6-27B / Gemma4-31B, vLLM, 2 H100s

III-4 · MORE THINKING ≠ MORE ACCURATE

accuracy fell in 21 of 36 comparisons

Holistic Agent Leaderboard: 9 benchmarks, 21,730 runs; raising the reasoning effort lowered accuracy in 21 of the 36 model–benchmark pairs while adding output tokens.

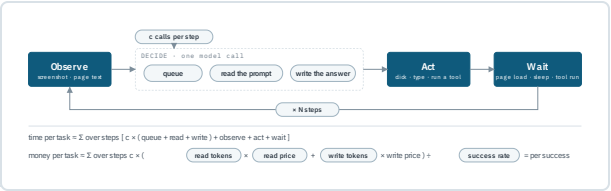
(Kapoor et al., 2025)

Raising the reasoning effort: 36 model–benchmark pairs, HAL



Kapoor et al., 2025 · 9 benchmarks, 21,730 runs

IV · The environment is slow; V · nothing overlaps



from the model call → here: the three environment terms and the serial structure → to: which term dominates

Four environment terms and one structural multiplier. Observation: building the accessibility tree (the operating system’s text list of on-screen elements). Execution: the browser and the page load. Fixed sleeps: the agent cannot tell when a page is ready. Tool tails: tests and builds. And all of it runs strictly in sequence.

IV-1 · OBSERVATION

3–26 s per accessibility tree

OSWorld desktop applications; the screenshot itself is under 2% of task time. In UFO2, OmniParser element detection adds about 1 s per step.

(Abhyankar, Qi & Zhang, 2026; Zhang et al., 2025)

IV-2, IV-3 · BROWSER AND FIXED SLEEPS

6.6 s browser vs 4.7 s model per step · 2–3 s sleep after every action

Medians over 151 WebVoyager live-site tasks with GPT-4o. OSWorld’s environment code sleeps 2 s after each action and OSWorld 2.0 prescribes 3 s: 318 steps ≈ 16 minutes of pure waiting (calc.); a single-call GUI agent (Kimi-K2.6) spends over 70% of its OSWorld time inside the desktop sandbox.

(Wong et al., 2026; xlang-ai, 2026; XLANG Lab, 2026; Chang et al., 2026)

IV-4 · TOOL TAILS

4% of tool calls take 85% of tool time

4,265 Claude Code and Codex sessions from 43 developers, September 2025 – June 2026: a request averages 4.3 minutes, of which tools take 2.5 (59.8%) and the model 1.7 (41.0%).

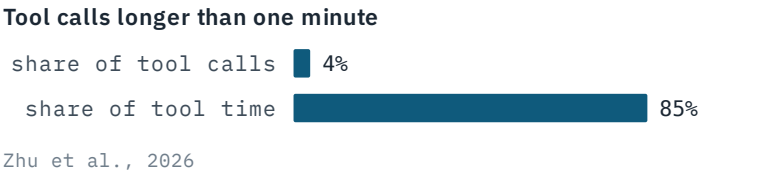
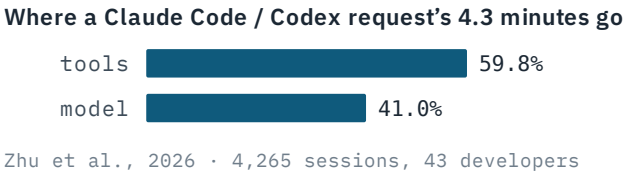
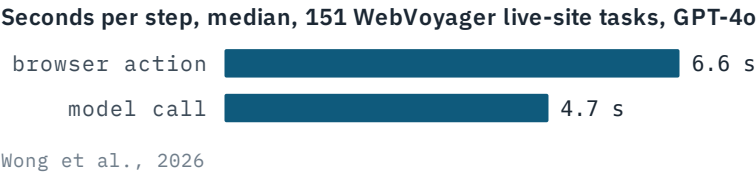
(Zhu et al., 2026)

V · SERIAL

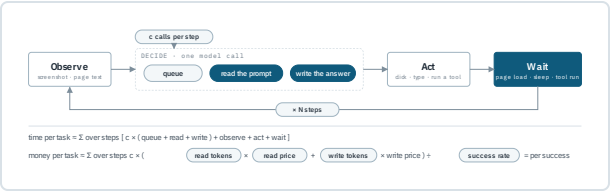
concurrency within a turn: 1.15

Copilot telemetry. Because the four stages never overlap, task time is the sum of every term above, not the largest one; fixing one term saves only its own share. Caution when reading “total time”: 80–92% of a coding session’s elapsed time is the human thinking between turns.

(Liu et al., 2026; Zhu et al., 2026)



Which term dominates depends on the harness, not on the kind of agent



from the five classes → here: the shares, per kind of agent → to: the six sources of cost

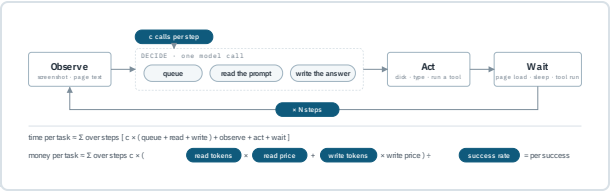
All five classes exist in every agent, but the heaviest term differs, and it flips when the harness changes. The one multiplier common to all three kinds is the number of passes, N.

	SCREENSHOT DESKTOP AGENT, MULTI-CALL HARNESS	TEXT WEB AGENT	CODING AGENT WITH A PROMPT CACHE
HEAVIEST	Reading × calls per step × steps. 87–97% of task time in planning, judging and reflection calls; screenshots and actions together under 3.5% (GTA1 and Agent S2 on 39 OSWorld tasks) (Abhyankar, Qi & Zhang, 2026)	Browser execution and waiting. Median per step: browser action 6.6 s, model call 4.7 s (151 WebVoyager live-site tasks, GPT-4o) (Wong et al., 2026)	Writing, or tool tails, depending on the tools. Token-by-token generation is 91–98.6% of model time (Yuan et al., 2026); Claude Code / Codex requests: tools 59.8%, model 41.0% (Zhu et al., 2026); Copilot: model 13.7%, tools 2% (Liu et al., 2026)
SECOND	Writing: thinking tokens lengthen every call	Reading, once the observation grows: 4.8× input moved the model's share from 46.9% to 61.6% (Chang et al., 2026); the Browser-Use harness spends 73% of its latency in model calls (Winston et al., 2026)	Reading, when load evicts the cache: request latency up to 7.14× (Kang et al., 2026)
NEGLIGIBLE	Observation and action: under 3.5% together (Abhyankar, Qi & Zhang, 2026)	Nothing: model and browser are close; which is larger depends on observation size and model speed	Navigation-only steps: there are no pages to turn
COUNTER-EXAMPLE	A single-call GUI agent spends over 70% of its time in the sandbox, because the benchmark sleeps 2–3 s after every action (Chang et al., 2026; xlang-ai, 2026; XLANG Lab, 2026)	Browser-Use: 73% of latency in the model, not the browser (Winston et al., 2026)	The two production records point opposite ways — heavy tools (tests, builds) vs light tools (file reads); the difference is the tools, not a contradiction
IN ONE LINE	Slow in reading: every pass re-reads the growing screenshot history	Slow in the environment, until the observation grows — then slow in reading	Reading is cached away; slow in writing and in the tools

1 · The harness sets the dominant term — calls per step, observation size, fixed sleeps, cache on or off. “Agents are slow because inference is slow” holds only for multi-call screenshot agents.

2 · N multiplies every term — one pass fewer saves a whole pass of time and money in all three kinds; it is the only lever common to all of them, and the reason OSWorld 2.0’s authors list “fewer environment rounds” as a goal in its own right.

Cost has six sources



from where the time goes → here: every cause of cost, in six classes → to: one class per page

Money comes from two things only: tokens read and tokens written, each times a price. Three ratios to remember — writing costs 5× reading; a cached read costs 0.1×; fast mode costs 2× (Anthropic, 2026b; OpenAI, 2026). Three of the remaining sources are multipliers on those two tokens; the last is the one nobody counts.

money per task =

$\Sigma \text{ over } N \text{ steps} \cdot c \times$
3 · NUMBER OF CALLS

read tokens
1 · TOKENS READ

+ write tokens × 5
2 · TOKENS WRITTEN

× price
5 · UNIT PRICE

÷ success rate
4 · FAILURES, RETRIES

+ machine hours
6 · THE MACHINES

1 · TOKENS READ

quadratic in the steps

History re-sent every call; 1,000–1,800 tokens per screenshot; rich observations 4.8×; sampling 96K → 920K tokens; and a cache that keeps 95.7% of the prefix still leaves prefix tokens at 59.5% of the bill.

(Abhyankar, Qi & Zhang, 2026; Anthropic, 2026a; Chang et al., 2026; Lee et al., 2026; Zhu et al., 2026)

2 · TOKENS WRITTEN

5× the price, billed even when it is thinking

224K output tokens per task (Claude Opus 4.8) vs 37K (GPT-5.5) on the same tasks; yet in an uncached screenshot agent output was only 31% of the total bill.

(XLANG Lab, 2026; Abhyankar, Qi & Zhang, 2026)

3 · NUMBER OF CALLS

N × c multiplies both

5–13× the calls of a single-call harness per step (GTA1, calc.); sampling 5–20 candidates per step multiplies calls by the candidate count.

(Abhyankar, Qi & Zhang, 2026; Lee et al., 2026)

4 · FAILURES AND RETRIES

\$72.4 per attempt → ≈ \$351 per success

Cost per success = cost per attempt ÷ success rate: OSWorld 2.0’s best agent completes 20.6% of tasks (calc.; an accounting conversion). Idle loops are billed in full: one loop, \$8.47.

(XLANG Lab, 2026; Abhyankar, Qi & Zhang, 2026)

5 · UNIT PRICE

\$4 / \$20 vs \$0.10 / \$0.50 per million tokens

Claude Opus 5.5 vs GPT-6 Luna, read / write; fast mode 2×; cache write 1.25×, cache read 0.1×; OpenAI charges 2× the read price above 272K tokens.

(Anthropic, 2026b; OpenAI, 2026)

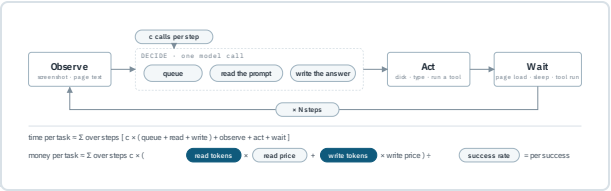
6 · THE MACHINES

0.3–4.6% of the API bill (CPU environments, calc.)

No benchmark reports a dollar figure for the sandbox, browser or VM; computed from hourly list prices, a CPU environment for a one-hour task costs \$0.02–0.36.

vendor price pages, read 2026-09-27

1–2 · Tokens read and tokens written



from the six classes → here: the two token bills → to: the three multipliers

Reading is the larger bill in a screenshot agent; writing is the dearer token. The read bill grows with the square of the steps and is not removed by caching; the write bill is inflated by thinking modes, which are billed as output.

READ · WHY IT GROWS

quadratic history · 1,000–1,800 tokens per screenshot · 4.8× rich observations · 9.6× sampling

Every call re-reads the k–1 earlier screenshots; 100 screenshots fill a 200K-token context; accessibility tree + HTML + screenshot multiplies the input 4.8×; 10 candidates per step take tokens per task from 96K to 920K.

(Abhyankar, Qi & Zhang, 2026; Anthropic, 2026a; Chang et al., 2026; Lee et al., 2026)

READ · CACHING IS NOT A CURE

95.7% hit rate, yet prefix tokens are 59.5% of the bill

4,265 Claude Code / Codex sessions: the prefix cache hits 95.7% of the time, but each step reads a history of a hundred thousand-plus tokens, so even at 0.1× the price the prefix is 59.5% of the list-price cost; on a miss the re-read is 3.8× the genuinely new content.

(Zhu et al., 2026)

WRITE · THINKING IS BILLED AS OUTPUT

224K vs 37K output tokens per task · but only 31% of an uncached bill

Same 108 OSWorld 2.0 tasks, Claude Opus 4.8 vs GPT-5.5; on Claude Opus 5.5 “thinking cannot be disabled and is billed as output”. In the uncached GTA1 runs the paper’s \$2.43 per task counts output only; all tokens from its Table 3 give \$7.87, so output was 31% of the total.

(XLANG Lab, 2026; Anthropic, 2026b; Abhyankar, Qi & Zhang, 2026)

One uncached screenshot agent’s bill per task, GTA1, 39 OSWorld tasks



Abhyankar, Qi & Zhang, 2026 · o3 list price, no cache

Input tokens per call as the observation grows, WebArena agent



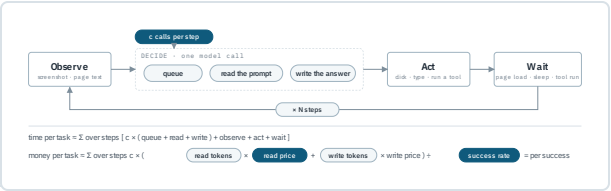
Chang et al., 2026

Share of list-price cost at a 95.7% prefix-cache hit rate



Zhu et al., 2026 · Claude Code / Codex sessions

3–5 · Calls, failures, and the price list



from the token bills → here: what multiplies them → to: four ways of counting the same task

Three multipliers sit in front of the token bills. The number of calls ($N \times c$) is the same quantity that made the agent slow; failures turn “per attempt” into “per success”; and the price list decides the constant — including what a speed tier or a long context costs.

3 · CALLS

5–13× per step, before sampling

GTA1 makes 4–12 planning calls per judging call, so one step costs 5–13× the calls of a single-call harness (calc.); sampling 5–20 candidates per step multiplies the calls by the candidate count. The same extra calls cost time and money at once.

(Abhyankar, Qi & Zhang, 2026; Lee et al., 2026)

4 · FAILURES

≈ \$351 per success · one dead loop \$8.47 · evaluation ≈ \$40,000

OSWorld 2.0’s best agent, Claude Opus 4.8: about \$72.4 per attempt at 20.6% completion, so ≈ \$351 per success (calc.; a conversion, not the price of retrying one task until it succeeds). Failed and idle steps are billed in full. Holistic Agent Leaderboard spent about \$40,000 on 21,730 runs across 9 benchmarks with a single run per configuration, and did not run Claude Opus 4.1 on Online-Mind2Web because the estimate was \$20,000.

(XLANG Lab, 2026; Abhyankar, Qi & Zhang, 2026; Kapoor et al., 2025)

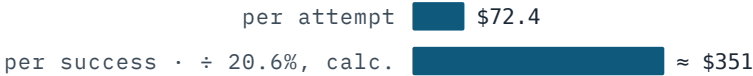
5 · THE PRICE LIST, SEPTEMBER 2026

\$4 / \$20 vs \$0.10 / \$0.50 per million tokens

Claude Opus 5.5 vs GPT-6 Luna, read / write. Fast mode: 2× on both vendors, and switching to it invalidates the cache. Cache write 1.25×, cache read 0.1×. OpenAI: 2× the read price for requests above 272K tokens.

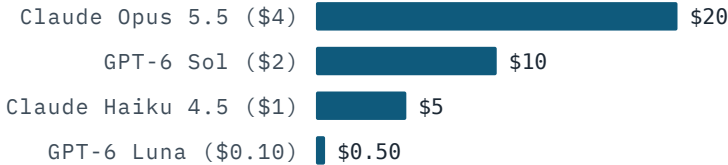
(Anthropic, 2026b; OpenAI, 2026)

Cost per attempt vs per success, OSWorld 2.0, Claude Opus 4.8



XLANG Lab, 2026 · 108 tasks

Write price per million tokens, September 2026 (read price in brackets)



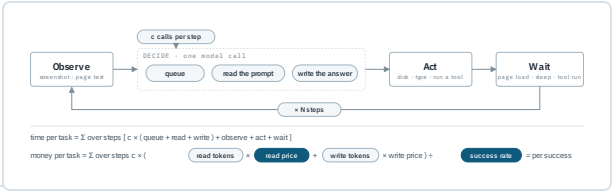
Anthropic, 2026b; OpenAI, 2026 · list prices

What evaluation costs, Holistic Agent Leaderboard



Kapoor et al., 2025 · 9 benchmarks, one run per configuration; Claude Opus 4.1 on Online-Mind2Web was skipped

Four ways of counting the same task, up to 30× apart

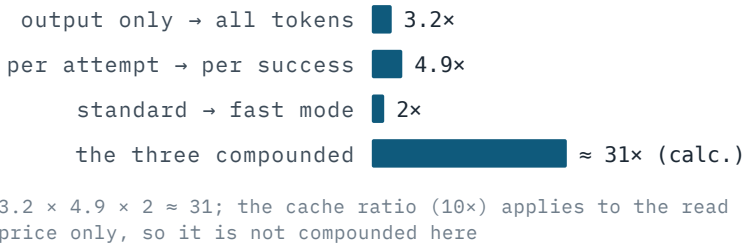


from the multipliers → here: the accounting conventions → to: the machines nobody counts

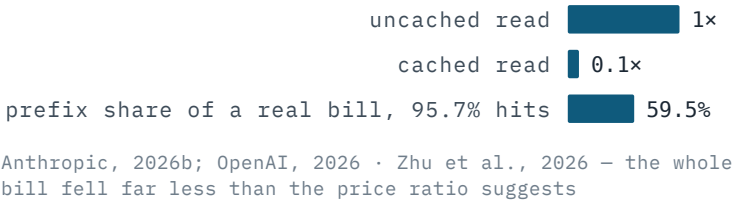
“What does one task cost?” has four honest answers, and they differ by up to 30×. The first question back is always: under which convention?

CONVENTION	THE SAME RUN, COUNTED BOTH WAYS	GAP
OUTPUT ONLY VS ALL TOKENS	GTA1 harness (o3 plans and judges; o3 list price \$2 read / \$8 write per million tokens, no cache) on 39 OSWorld tasks: the paper’s text says \$2.43 per task, which counts output tokens only; the input and output tokens of its Table 3 give \$7.87 (Abhyankar, Qi & Zhang, 2026)	3.2× — same run, same tasks
PER ATTEMPT VS PER SUCCESS	OSWorld 2.0, 108 long tasks, Claude Opus 4.8 with batched actions: about \$72.4 per attempt at 20.6% completion; \$72.4 ÷ 20.6% ≈ \$351 (calc.) (XLANG Lab, 2026)	4.9× — an accounting conversion, not the cost of retrying one task to success
UNCACHED VS CACHED	A cached read costs 0.1× on both price lists (Anthropic, 2026b; OpenAI, 2026); but in 4,265 Claude Code / Codex sessions the prefix cache hit 95.7% and prefix tokens were still 59.5% of the list-price cost, because each step reads a hundred thousand-plus tokens of history (Zhu et al., 2026). Screenshot-agent measurements do not report their cache state at all	10× on the read price; the saving on the whole bill is far smaller and depends on the hit rate
STANDARD VS FAST MODE	Anthropic fast mode: Claude Opus 5.5 from \$4 / \$20 to \$8 / \$40, Claude Opus 5 and 4.8 from \$5 / \$25 to \$10 / \$50 per million tokens; up to 2.5× faster output (vendor-stated), no change to the wait for the first token, and switching invalidates the cache. OpenAI: 2× on every listed model; “up to 2.5×” speed is stated only for GPT-5.6 Sol (Anthropic, 2026b; OpenAI, 2026)	2× — same model, same tokens; only the writing speed is bought

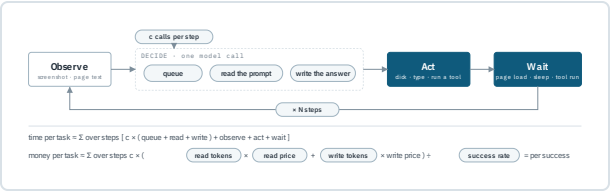
The gap each convention opens on the same run



Read price, cached vs uncached, and what it saved in practice



6 · The machines: not in any benchmark’s dollar figure, but computable



from the API bill → here: the environment’s own bill → to: how slow and expensive are linked

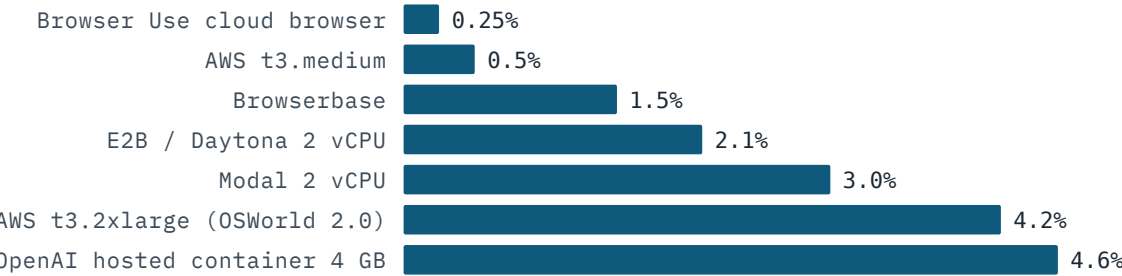
Every published “cost per task” is an API bill. No benchmark reports a dollar figure for its sandbox, browser or virtual machine, and no product publishes a per-session environment cost. From hourly list prices (read 2026-09-27) it can be computed: a one-hour task on a CPU environment costs \$0.02 (Browser Use cloud browser) to \$0.36 (OpenAI hosted container, 4 GB) — 0.3–4.6% of a \$7.87 API bill and 0.03–0.5% of a \$72.4 one (calc.).

FOUR EXCEPTIONS THAT MAKE IT LARGE

Plan fees — Browserbase \$20 per month for 100 hours, E2B Pro \$150 per month: expensive per task when the hours go unused. **Traffic** — residential proxies \$5–12 per GB; 1 GB equals 40 browser-hours on Browser Use. **Wall-clock billing while the agent waits** — in AgentSysBench’s production trace the median session is active only 20% of its lifetime. **GPU sandboxes** — Daytona’s H100 at \$3.95 per hour is half of a \$7.87 bill in one hour; in AgentSysBench’s simulation workload the GPU container is over 99% of per-request cost.

vendor price pages; Chang et al., 2026

A one-hour CPU environment as a share of a \$7.87 API bill (calc.)



vendor price pages, read 2026-09-27 · against \$72.4 the shares are 0.03–0.50%

HOW BENCHMARKS HANDLE IT

instance types, never dollars

OSWorld, OSWorld-Verified, OSWorld 2.0, TheAgentCompany and WebArena all name their machines (t3.medium / t3.large controllers, t3.2xlarge, t3a.xlarge with a 1,000 GB disk for the hosted sites) and none gives their cost. OSWorld 2.0’s \$72.4 moves with output tokens, so it is an API bill; the Holistic Agent Leaderboard defines “Cost” as total API cost. Only AgentSysBench itemises the sandbox — as a share, not a dollar.

(XLANG Lab, 2026; Kapoor et al., 2025; Chang et al., 2026)

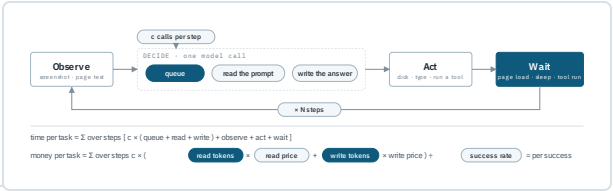
HOW PRODUCTS HANDLE IT

a dedicated machine per user, price undisclosed

Meta Muse runs on “a dedicated cloud computer with its own browser”; OpenAI’s ChatGPT agent uses “its own virtual computer” (Operator, “its own browser”, was retired on 2025-08-31); Claude in Chrome runs inside the user’s own Chrome. Subscription prices are not environment costs.

(Meta, 2026; OpenAI, 2025)

How slow and expensive are linked: three kinds of relationship



from the two bills → here: what each cause does to time and to money → to: what fixing each lever buys

The two equations share the read and write tokens; environment time and queueing appear only in the time equation. So every cause relates to time and money in exactly one of three ways.

SAME SOURCE — IN BOTH EQUATIONS

fix it, and time and money fall together

Steps N: time roughly linear, read tokens quadratic — double the steps, double the time, about 4× the read bill (uncached). **Calls per step. Reading:** history, screenshots, rich observations. **Writing:** thinking — the slowest and the dearest token. **Cache hits:** save both, but fragile — any edit to the history or a switch to fast mode invalidates them. **Failures and idle loops:** both, and they turn “per attempt” into “per success”.

(Abhyankar, Qi & Zhang, 2026; Anthropic, 2026b)

SLOW BUT NOT EXPENSIVE — TIME EQUATION ONLY

fix it, and only time falls

Queueing. Page loads and fixed sleeps. Tool tails (tests, builds). None of these produces a token. The machine time behind them is billed by the hour: CPU sandboxes \$0.02–0.36 per hour, usually under 5% of the API bill (page 15). To escape the queue one buys a fast or priority tier — at 2× the price.

MONEY FOR TIME OR FOR ACCURACY — OPPOSITE SIGNS

buying speed buys only the writing segment

Fast mode: up to 2.5× faster writing (vendor-stated), reading and queueing unchanged, 2× the price. **Bigger models, more thinking:** slower and dearer, and the cost-versus-accuracy curve is convex — each extra point costs more than the last: on one model the marginal tokens per point rise from 101K to 575K; 6×, 9× and 9.6× the tokens for a few points; and more thinking is not always more accurate. **Smaller models:** faster and cheaper, but if accuracy falls, steps and retries rise — possibly slower and dearer again (inference; no direct evidence).

(Anthropic, 2026b; OpenAI, 2026; XLANG Lab, 2026; Kapoor et al., 2025; Lee et al., 2026)

TIME

N steps

c calls

×

queue

+

read

+

write

)+

observe

+

act

+

wait

MONEY

N steps

c calls

×

read tokens

×

price

+

write tokens

×

5

×

price

)+

success rate

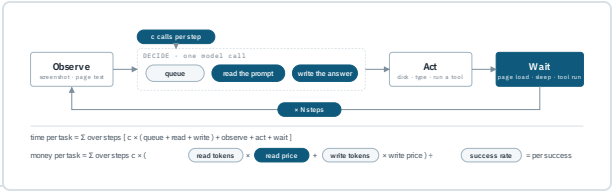
in both equations — same source

time equation only — slow but not expensive

the price — money for time or accuracy

In one line: a cause that lives in tokens — fix it and time and money fall together; a cause that lives in waiting — fix it and only time falls; paying for speed buys only the writing segment.

What fixing each lever buys

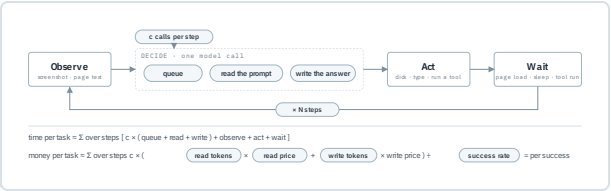


from the three relationships → here: the levers, read off the equations → to: what has not been measured

Seven levers follow from the equations; each one saves a known term and carries a known risk. This table is the bridge to Part 2, which asks how far the literature has pushed each lever.

LEVER	TIME	MONEY	THE CATCH
Fewer passes — fewer steps, several actions per call, skip navigation-only steps	yes	yes, and it is the quadratic term	one must know which steps need no thinking
Read less — trim the history, shrink screenshots, cache the prefix	yes	yes	trim too much and the agent forgets; trimming the history itself invalidates the cache
Write less — turn thinking down, use a smaller model	yes	yes	accuracy may fall
Fewer calls per step — drop judging and reflection calls	yes	yes	tasks that relied on those calls to catch errors get worse
Overlap the waiting — pre-load, run tools in parallel, act on page-ready events instead of fixed sleeps	yes	no	helps only where the environment is the dominant term
Buy a fast or priority tier	yes, writing only	no — costs 2× more	reading is not faster; switching drops the cache
Stop retrying failures — early stop, detect dead loops	yes	yes	kills some attempts that would have succeeded

What nobody has measured yet

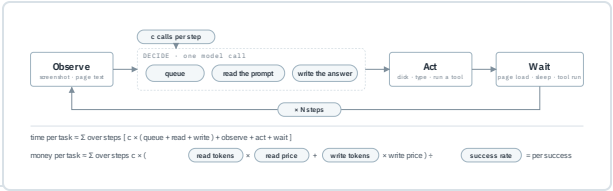


from the levers → here: the holes in the evidence → to: references

These are gaps in the measurements, not claims of a research gap. They decide how far each statement in this part can be pushed.

NOT MEASURED	WHAT EXISTS	WHAT IT STOPS US FROM SAYING
Read / write time split and cache hit rate for a frontier API model	No measurement reports both. The only read / write split is on locally served 27–31B models (Yuan et al., 2026); screenshot-agent measurements do not report cache state	How much of “87–97% in the model” is reading, how much writing, and what remains once the cache is on
How much slower than a person	None of 17 benchmarks times agent and human on the same tasks; the closest is AXIS’s user study, where a UI agent was 1.69× slower than manual work on easy tasks — small sample (Lu et al., 2025)	“X times slower than a person” cannot go on a slide
Environment dollars	Benchmarks report instance types, not dollars; AgentSysBench gives shares. Computed from list prices: usually 0.3–5% of the API bill for CPU environments (page 15)	Every published cost is an API bill; the omission is small for CPU environments, not for GPU sandboxes or long idle sessions
Whether OSWorld-Human’s “action” time includes the 2-s sleep	Not stated; if it does, “actions under 2%” implies more than 100 s per step (calc.)	“Environment under 3.5%” may understate the waiting
The rebound from a smaller model	No direct evidence on how many extra steps and retries a drop in accuracy causes	The last row of the relationship table stays marked “inference”
How much slowness costs accuracy	OSWorld 2.0 records the stale-state failure mode but not its share of failures (XLANG Lab, 2026)	“It exists” is all that can be said, not “how large”

References · Part 1 (1 of 2)



author-year tags on the pages refer to these entries · published version where one exists, otherwise the arXiv number and the authors’ institutions

1. Abhyankar, R., Qi, Q., & Zhang, Y. (2026). OSWorld-Human: Benchmarking the efficiency of computer-use agents. *Proceedings of the 9th Conference on Machine Learning and Systems (MLSys 2026)*. arXiv:2506.16042. University of California, San Diego.

2. Anthropic. (2026a, May 13). *Best practices for computer and browser use with Claude*. <https://claude.com/blog/best-practices-for-computer-and-browser-use-with-claude>

3. Anthropic. (2026b). *Claude API pricing; Fast mode* (developer documentation, read 27 September 2026). <https://platform.claude.com/docs/en/about-claude/pricing>; <https://platform.claude.com/docs/en/build-with-claude/fast-mode>

4. Artificial Analysis. (2026). *Claude Opus 5.5 (high): model page* and *Methodology* (read September 2026). <https://artificialanalysis.ai/models/claude-opus-5-5-high>; <https://artificialanalysis.ai/methodology>

5. Bian, S., Yan, M., Jayarajan, A., Pekhimenko, G., & Venkataraman, S. (2025). What limits agentic systems efficiency? arXiv:2510.16276. University of Wisconsin–Madison; University of Toronto; NVIDIA.

6. Chang, C., Zhou, Y., Fu, K., An, D., Feng, T., Lu, H., Yao, S., Guo, P., Yu, Y., Shan, Y., Li, B., Yuan, B., & Wang, W. (2026). From LLM inference to agentic workloads: Characterization and implications for serving systems (AgentSysBench). arXiv:2608.15127. Hong Kong University of Science and Technology; Alibaba Group; ByteDance.

7. Kang, H., Li, Z., Xu, W., Yang, X., Chen, Y., Wang, J., Chen, B., Krishna, T., Xu, C., & Arora, S. (2026). ThunderAgent: A simple, fast and program-aware agentic inference system. *Proceedings of the 43rd International Conference on Machine Learning (ICML 2026)*, poster. arXiv:2602.13692. Georgia Institute of Technology; University of Illinois Urbana-Champaign; Carnegie Mellon University; Together AI.

8. Kapoor, S., Stroebl, B., Kirgis, P., Nadgir, N., Siegel, Z. S., Wei, B., ... Narayanan, A. (2025). Holistic Agent Leaderboard: The missing infrastructure for AI agent evaluation. arXiv:2510.11977. Princeton University et al.

9. Le Sellier De Chezelles, T., Gasse, M., Drouin, A., Caccia, M., Boisvert, L., Thakkar, M., ... Lacoste, A. (2025). The BrowserGym ecosystem for web agent research. arXiv:2412.05467. ServiceNow Research et al. (figure not re-verified; used only in Appendix A1)

10. Lee, N., Erdogan, L. E., John, C. J., Krishnapillai, S., Mahoney, M. W., Keutzer, K., & Gholami, A. (2026). Agentic test-time scaling for WebAgents. arXiv:2602.12276. University of California, Berkeley.

11. Liu, B., Qiu, H., Goiri, Í., Fonseca, R., Bianchini, R., & Choukse, E. (2026). Agentic coding in the wild: Characterizing GitHub Copilot at production scale. arXiv:2608.00101. University of Illinois Urbana-Champaign; Microsoft Azure Research.

12. Lu, J., Zhang, Z., Yang, F., Zhang, J., Wang, L., Du, C., Lin, Q., Rajmohan, S., Zhang, D., & Zhang, Q. (2025). AXIS: Efficient human-agent-computer interaction with API-first LLM-based agents. *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 7711–7743. <https://doi.org/10.18653/v1/2025.acl-long.381>. Microsoft.

References · Part 1 (2 of 2)

continued

13. Meta. (2026, September 8). *Introducing Muse, your personal AI agent* [Newsroom post]. <https://about.fb.com/news/2026/09/introducing-muse-personal-ai-agent/>

14. OpenAI. (2025). *Introducing Operator* (23 January 2025); *Introducing ChatGPT agent* (17 July 2025). <https://openai.com/index/introducing-operator/>; <https://openai.com/index/introducing-chatgpt-agent/>

15. OpenAI. (2026). *API pricing*; *API changelog* (fast mode, 30 July 2026; Ultrafast, 13 August 2026) (read 27 September 2026). <https://developers.openai.com/api/docs/pricing>; <https://developers.openai.com/api/docs/changelog>

16. Winston, C., Wang, R. Y., Mirhoseini, A., & Kozyrakis, C. (2026). Agent JIT compilation for latency-optimizing web agent planning and scheduling. *Proceedings of the 43rd International Conference on Machine Learning (ICML 2026)*, PMLR 306. arXiv:2605.21470. Stanford University.

17. Wong, M., Hsieh, K., Nath, S., & Netravali, R. (2026). Skim: Speculative execution for fast and efficient web agents. arXiv:2605.16565. Princeton University; Microsoft Research.

18. xlang-ai. (2026). *OSWorld* [Source code], `desktop_env/desktop_env.py` and `run.py` at commit b138d348. <https://github.com/xfang-ai/OSWorld>

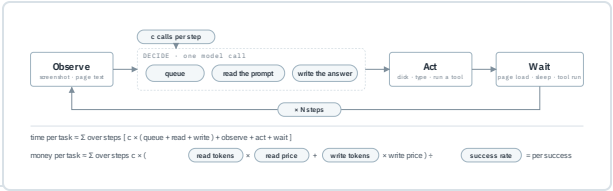
19. XLANG Lab. (2026). OSWorld 2.0: Benchmarking computer use agents on long-horizon real-world tasks. arXiv:2606.29537. The University of Hong Kong (authored as “XLANG Lab and Collaborators”; the full author list is in the paper’s Appendix A).

20. Yuan, Y., Nayak, A., Kundu, S., & Talati, N. (2026). Agentic AI workload characteristics. arXiv:2605.26297. University of Illinois Urbana-Champaign; Gimlet Labs; Intel.

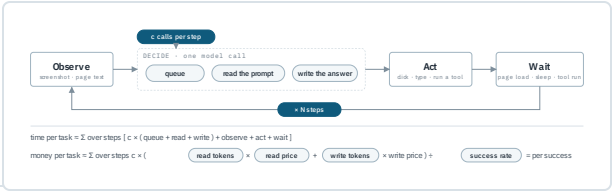
21. Zhang, C., Huang, H., Ni, C., Mu, J., Qin, S., He, S., ... Zhang, D. (2025). UFO2: The desktop AgentOS. arXiv:2504.14603. Microsoft.

22. Zhu, K., Jacob, M., Ma, C., Pan, Y., Wang, S., Krishnamurthy, A., & Kasikci, B. (2026). TraceLab: Characterizing coding agent workloads for LLM serving. arXiv:2606.30560. University of Washington.

23. Vendor price pages used for Appendix A3 (all read 27 September 2026): AWS, *Amazon EC2 T3 instances*; Browser Use, *Pricing* and *API v4: create browser session*; Browserbase, *Pricing* and *Billing plans*; Daytona, *Pricing* and *Billing*; E2B, *Pricing*; Modal, *Pricing* and *Sandbox resources*.



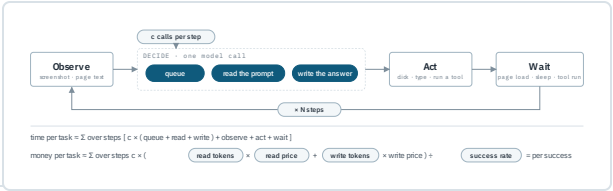
A1 · Slowness, every cause — I. steps and II. calls per step



the complete table behind pages 4–6 · three kinds of agent: screenshot desktop agent, text web agent, coding agent with a prompt cache

ID	CAUSE	MECHANISM	EVIDENCE, WITH THE CONDITIONS OF THE MEASUREMENT	SOURCE	DOMINANT IN
I-1	The task is long	Hour-scale tasks take hundreds of passes	OSWorld 2.0, 108 long tasks: Claude Opus 4.7 with maximum thinking, one action per step, 500-step cap: 318 tool calls per task on average, i.e. 318 steps; with several actions per call, Claude Opus 4.8 takes 103 steps (481.8 calls) and GPT-5.5 95.2 steps (149.8 calls)	(XLANG Lab, 2026)	screenshot, text
I-2	One action per pass	Every click is a whole pass: one observation, one model call, one wait	OSWorld 2.0: with one action per step Claude Opus 4.7 needs 318 steps; with batched actions the same model needs 160.7 and Claude Opus 4.8 needs 103	(XLANG Lab, 2026)	screenshot, text
I-3	Navigation-only passes still call the model	Paging, scrolling and opening the next page need no thinking but cost a call each	Skim’s profile of three text web agents (Browser-Use, AgentOccam, WebVoyager; GPT-4o) on 151 WebVoyager live-site tasks: 66.7% of the steps in the median task are pure navigation	(Wong et al., 2026)	text
I-4	Idling and dead loops	Element not found, repeated retries; steps rise without progress	OSWorld-Human’s records of the GTA1 harness (o3 plans and judges, GTA1-7B locates elements) on 39 OSWorld desktop tasks: in failed tasks that exceeded 50 steps, 66% of steps were wasted repeats; one element-locating loop repeated the same step 18 times — 27 minutes and \$8.47 at list price without caching	(Abhyankar, Qi & Zhang, 2026)	screenshot
I-5	Acting on a stale screen	The screenshot is taken before the page changes; the agent clicks on the old layout and must repair	OSWorld 2.0 records “stale interface state” as a failure mode; no share reported	(XLANG Lab, 2026)	screenshot
II-1	Multi-call harness	One step = plan + judge + reflect, several calls in series	OSWorld-Human, two harnesses: GTA1 runs 4 parallel planners per step, retries up to 3 rounds, then one judging call picks — 4–12 planning calls per judging call; Agent S2 adds a reflection call per step, so planning is 53% and reflection 34% of task time	(Abhyankar, Qi & Zhang, 2026)	screenshot
II-2	Sampling for accuracy	5–20 candidates per step, then pick one	Same model and harness (gpt-oss-120b, ReAct) on 165 WebArena-Lite tasks: 1 → 10 candidates per step raises success from 38.8% to 43.2% and tokens per task from 96K to 920K — 9.6× the tokens for 4.4 points	(Lee et al., 2026)	text
II-3	Coding agents call repeatedly within a turn	The model is called back and forth inside one user turn	GitHub Copilot coding agent production telemetry (first week of June 2026, 13.5 million sessions): 6.6 model calls per user turn on average	(Liu et al., 2026)	coding

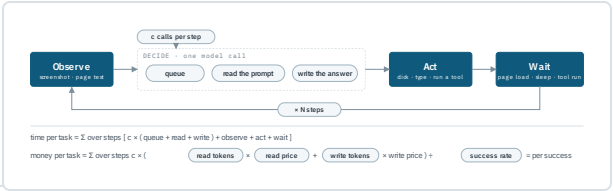
A1 · Slowness, every cause — III. each call is slow



the complete table behind pages 7–8

ID	CAUSE	MECHANISM	EVIDENCE, WITH THE CONDITIONS OF THE MEASUREMENT	SOURCE	DOMINANT IN
III-1	Queueing	The request waits at the provider; unbilled, but counted as model time	Client-side measurement of 15 models across 5 providers: requests of the same length differ in latency by up to 69× depending on when they are sent	(Bian et al., 2025)	all
III-2	Reading (the prompt is re-processed every call)	Every call re-reads the whole history; more screenshots, slower	OSWorld-Human, multi-call harnesses: later steps up to 3× slower because the prompt at step k holds the k–1 earlier screenshots — “dominated by prefill”. One screenshot is 1,000–1,800 tokens. AgentSysBench’s WebArena agent (Kimi-K2.6): observation switched from a single format to accessibility tree + HTML + screenshot → input 4.8×, model share of time 46.9% → 61.6%. Under load the cache is evicted and re-read: SWE-Agent / OpenHands running GLM-4.6 on 8 H100s for SWE-bench Lite, request latency up to 7.14× as concurrency rises	(Abhyankar, Qi & Zhang, 2026; Anthropic, 2026a; Chang et al., 2026; Kang et al., 2026)	screenshot; text with rich observations; coding under load
III-3	Writing (one token at a time)	Token-by-token generation, strictly serial	Locally served ReAct agents (Qwen3.6-27B / Gemma4-31B, vLLM, 2 H100s) on five benchmarks: with the cache warm, generation is 91–98.6% of model time. Windows desktop agent UFO2 (GPT-4o / o1 API): about 10 s per model call, the largest item per step in every configuration	(Yuan et al., 2026; Zhang et al., 2025)	coding; single-call screenshot
III-4	Thinking modes and larger models	Thinking writes more tokens; larger models are slower per token	OSWorld 2.0, same 108 tasks: Claude Opus 4.8 writes 224K output tokens per task, GPT-5.5 37K. Holistic Agent Leaderboard, 9 benchmarks, 21,730 runs: raising reasoning effort lowered accuracy in 21 of 36 pairs	(XLANG Lab, 2026; Kapoor et al., 2025)	all

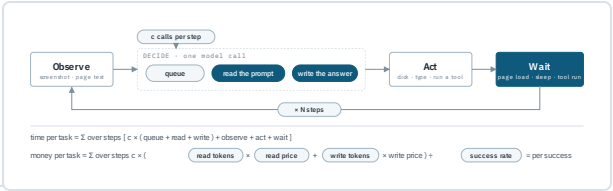
A1 · Slowness, every cause — IV. environment, V. serial



the complete table behind page 9

ID	CAUSE	MECHANISM	EVIDENCE, WITH THE CONDITIONS OF THE MEASUREMENT	SOURCE	DOMINANT IN
IV-1	Producing the observation	Extracting the accessibility tree or detecting elements takes time	OSWorld desktop applications: 3–26 s to generate one accessibility tree; the screenshot itself is under 2% of task time. UFO2: OmniParser element detection adds about 1 s per step	(Abhyankar, Qi & Zhang, 2026; Zhang et al., 2025)	screenshot
IV-2	Browser execution and page load	After a click the agent waits for the page	151 WebVoyager live-site tasks: per step, median browser action 6.6 s vs model call 4.7 s. WebArena with GenericAgent + Claude 3.5 Sonnet: 7.6 s of a 12.2 s step in the browser (this figure was not re-verified)	(Wong et al., 2026; Le Sellier De Chezelles et al., 2025)	text
IV-3	Fixed sleeps	The agent cannot tell when the page is ready, so it sleeps a fixed number of seconds	OSWorld’s environment code sleeps 2 s after every action; OSWorld 2.0 prescribes 3 s, so 318 steps are about 16 minutes of pure waiting (calc.). AgentSysBench: a single-call GUI agent (ReAct, Kimi-K2.6 API) spends over 70% of its OSWorld execution time in the desktop sandbox	(xlang-ai, 2026; XLANG Lab, 2026; Chang et al., 2026)	screenshot (caused by the benchmark harness)
IV-4	Tool tails	Tests and builds take minutes	TraceLab, 4,265 Claude Code / Codex sessions from 43 developers (September 2025 – June 2026): tool calls over 1 minute are 4% of calls but 85% of tool time; a request averages 4.3 minutes — tools 2.5 (59.8%), model 1.7 (41.0%)	(Zhu et al., 2026)	coding
V	Serial structure	The four stages never overlap, so task time is the sum of every term, not the largest one; fixing one term saves only its share	Concurrency within a turn is 1.15 in Copilot telemetry. Separately, 80–92% of a coding session’s elapsed time is the human thinking between turns — strip it before reading any total	(Liu et al., 2026; Zhu et al., 2026)	all

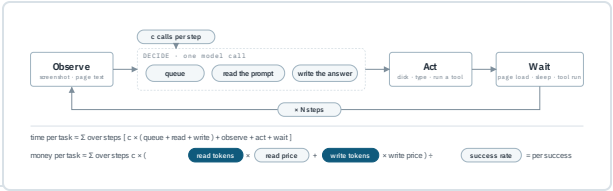
A2 · Where the arrows land, with every measurement condition



the complete table behind page 10

	SCREENSHOT DESKTOP AGENT, MULTI-CALL HARNESS	TEXT WEB AGENT	CODING AGENT WITH A PROMPT CACHE
heaviest term	Reading × calls per step × steps. OSWorld-Human timed two multi-call harnesses (GTA1, Agent S2) step by step on 39 OSWorld desktop tasks: 87–97% of task time in planning, judging and reflection calls; screenshots and actions together under 3.5% (Abhyankar, Qi & Zhang, 2026)	Browser execution plus waiting. Skim measured three text web agents (GPT-4o) on 151 WebVoyager live-site tasks: per step, median browser action 6.6 s and model call 4.7 s (Wong et al., 2026)	Writing, or tool tails, depending on the tools. With the context cache on, token-by-token generation is 91–98.6% of model time (locally served models) (Yuan et al., 2026); in Claude Code / Codex records a request spends 59.8% in tools and 41.0% in the model (Zhu et al., 2026); in GitHub Copilot, whose tools are light, agent time is 13.7% model and 2% tools (Liu et al., 2026)
second	Writing: with thinking on, more reasoning tokens per step, and generation is one token at a time	Reading, once the observation grows: AgentSysBench switched a WebArena agent from one observation format to accessibility tree + HTML + screenshot — input 4.8×, model share 46.9% → 61.6% (Chang et al., 2026); the Browser-Use harness spends 73% of its latency in model calls (Winston et al., 2026)	Reading, when load evicts the cache and the whole context is re-read: ThunderAgent measured request latency up to 7.14× under cache thrash (Kang et al., 2026)
negligible	Producing the observation and executing the action: under 3.5% together (Abhyankar, Qi & Zhang, 2026)	Nothing: model and browser are close; which is larger depends on observation size and model speed	Navigation-only steps: a coding agent has no pages to turn
counter-example	A single-call agent is different: AgentSysBench’s one-call GUI agent (Kimi-K2.6) spends over 70% of its OSWorld time in the desktop sandbox (Chang et al., 2026), because the harness sleeps after every action — 2 s in OSWorld’s code (xlang-ai, 2026), 3 s in OSWorld 2.0 (XLANG Lab, 2026)	Browser-Use: 73% of latency in model calls, not the browser (Winston et al., 2026)	Two production records point opposite ways: heavy tools in Claude Code / Codex (tools 59.8% > model 41.0%) (Zhu et al., 2026); light tools in Copilot (model 13.7% > tools 2%) (Liu et al., 2026). The difference is the kind of tool (tests and builds vs file reads), not a contradiction
in one line	Slow in reading: every step re-reads the screenshot history, more as the task goes on	Slow in the environment (browser and page waits); once the observation grows, slow in reading again	Reading is cached away; slow in writing and in the tools

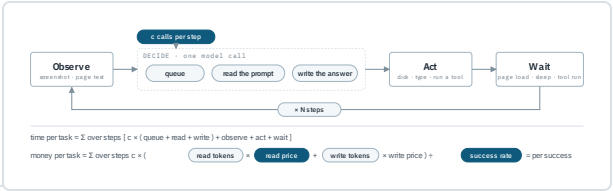
A3 · Cost, every cause — 1. tokens read and 2. tokens written



the complete table behind pages 11–12 · prices are list prices from the vendors’ own pages, September 2026

ID	CAUSE	MECHANISM	EVIDENCE, WITH THE CONDITIONS OF THE MEASUREMENT	SOURCE
1-1	History re-sent	Call k reads everything from the k–1 earlier steps; total reading ≈ steps squared	“Cost grows quadratically with the number of steps” — OSWorld-Human, real runs	(Abhyankar, Qi & Zhang, 2026)
1-2	Large screenshots	1,000–1,800 tokens per image; 100 images fill a 200K context	Anthropic’s engineering guidance for computer and browser use	(Anthropic, 2026a)
1-3	Rich observation formats	Accessibility tree + HTML + screenshot together: 4.8× the input	AgentSysBench, WebArena agent, Kimi-K2.6	(Chang et al., 2026)
1-4	Sampling	Several candidates per step, mostly read tokens	gpt-oss-120b + ReAct on 165 WebArena-Lite tasks: 1 → 10 candidates per step, 96K → 920K tokens per task	(Lee et al., 2026)
1-5	The cache is not a cure	However high the hit rate, a long history keeps reading dominant; a miss re-reads everything	TraceLab, 4,265 Claude Code / Codex sessions: prefix-cache hit rate 95.7%, prefix tokens still 59.5% of list-price cost; on a miss the re-read is 3.8× the genuinely new content	(Zhu et al., 2026)
2-1	Thinking modes	Thinking tokens are billed as output; when they cannot be switched off they are a fixed overhead	OSWorld 2.0, same tasks: Claude Opus 4.8 224K output tokens per task, GPT-5.5 37K; Claude Opus 5.5: “thinking cannot be disabled and is billed as output”	(XLANG Lab, 2026; Anthropic, 2026b)
2-2	But writing is not always the larger bill	In an uncached screenshot agent, output was 31% of the total	OSWorld-Human, GTA1: the paper’s \$2.43 counts output only; Table 3’s input and output give \$7.87	(Abhyankar, Qi & Zhang, 2026)

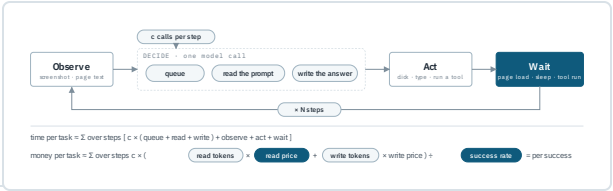
A3 · Cost, every cause — 3. calls, 4. failures, 5. unit price



the complete table behind page 13

ID	CAUSE	MECHANISM	EVIDENCE, WITH THE CONDITIONS OF THE MEASUREMENT	SOURCE
3	Number of calls	Steps × calls per step scales both token bills	GTA1: 4 parallel planners per step, up to 3 retry rounds, one judging call → 4–12 planning calls per judging call, i.e. 5–13× the calls of a single-call harness (calc.). Agentic test-time scaling: 5–20 candidates per step multiply the calls by the candidate count. The same extra calls cost time and money at once	(Abhyankar, Qi & Zhang, 2026; Lee et al., 2026)
4-1	Failed attempts are billed	Cost per success = cost per attempt ÷ success rate	OSWorld 2.0: the best agent (Claude Opus 4.8) costs about \$72.4 per attempt at 20.6% completion → about \$351 per success (calc.). An accounting conversion, not the real price of retrying one task until it succeeds	(XLANG Lab, 2026)
4-2	Idle steps are billed in full	Every pass of a dead loop is a full read and write	OSWorld-Human, GTA1 on 39 OSWorld tasks: one element-locating loop repeated the same step 18 times — 27 minutes, \$8.47 at list price without caching; in failed tasks over 50 steps, 66% of steps were such repeats	(Abhyankar, Qi & Zhang, 2026)
4-3	Evaluation itself is too expensive to repeat	Single runs, no repeats	Holistic Agent Leaderboard: 9 benchmarks, 21,730 runs, one run per configuration, about \$40,000 in total; Claude Opus 4.1 not run on Online-Mind2Web because the estimate was \$20,000	(Kapoor et al., 2025)
5-1	Model tier	Claude Opus 5.5 \$4 / \$20 vs GPT-6 Luna \$0.10 / \$0.50 per million tokens read / written	Vendor price pages, 27 September 2026	(Anthropic, 2026b; OpenAI, 2026)
5-2	Fast mode	2× on both vendors; switching speed tiers invalidates the cache	Anthropic: Claude Opus 5.5 \$8 / \$40, up to 2.5× output speed (vendor-stated), first-token wait unchanged. OpenAI: 2× on all listed models; “up to 2.5×” stated only for GPT-5.6 Sol	(Anthropic, 2026b; OpenAI, 2026)
5-3	Cache write and read	Writing to the cache 1.25×; reading from it 0.1×	Vendor price pages	(Anthropic, 2026b; OpenAI, 2026)
5-4	Long-context surcharge	OpenAI: 2× the read price above 272K tokens	Vendor price page	(OpenAI, 2026)

A3 · Cost — 6. the machines, and the four conventions

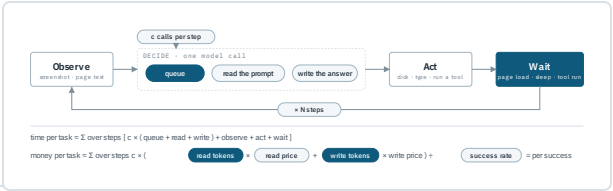


the complete tables behind pages 14–15 · vendor price pages read 2026-09-27 · per-task amounts and shares are our own arithmetic (calc.)

ENVIRONMENT	PRICE (VENDOR PAGE)	1-HOUR TASK	15-MIN TASK	SHARE OF A \$7.87 API BILL	SHARE OF \$72.4
Browser Use cloud browser	\$0.02 per browser-hour, billed by the minute, 1-minute minimum; traffic extra (residential proxy \$5/GB, direct \$0.20/GB); sessions capped at 240 minutes	\$0.020	\$0.005	0.25%	0.03%
AWS t3.medium (2 vCPU / 4 GiB, Linux, us-east-1)	\$0.0418 per hour on the T3 product page; another AWS page says \$0.0416 (0.5% apart)	\$0.042	\$0.010	0.5%	0.06%
Browserbase cloud browser	\$20 per month for 100 hours, then \$0.12 per hour; \$99 per month for 500 hours, then \$0.10; billed by the minute	\$0.12	\$0.03	1.5%	0.17%
E2B / Daytona sandbox (2 vCPU / 4 GiB)	E2B \$0.000014 per vCPU-second + \$0.0000045 per GiB-second; Daytona \$0.0504 per vCPU-hour + \$0.0162 per GiB-hour — both \$0.1656 per hour	\$0.166	\$0.041	2.1%	0.23%
Modal sandbox (1 physical core = 2 vCPU, 4 GiB)	\$0.00003942 per core-second + \$0.00000667 per GiB-second → \$0.238 per hour; billed on the larger of requested and used	\$0.238	\$0.060	3.0%	0.33%
AWS t3.2xlarge (8 vCPU / 32 GiB, OSWorld 2.0's default instance)	\$0.3341 per hour	\$0.334	\$0.084	4.2%	0.46%
OpenAI hosted container (Hosted Shell / Code Interpreter, 4 GB)	\$0.12 per 20-minute session → \$0.36 per hour; 1 GB \$0.03, 16 GB \$0.48, 64 GB \$1.92 per 20 minutes	\$0.36	\$0.12	4.6%	0.50%

The four accounting conventions (page 14), in one line each: output only vs all tokens — 3.2× (OSWorld-Human, \$2.43 vs \$7.87); per attempt vs per success — 4.9× (OSWorld 2.0, \$72.4 ÷ 20.6%); uncached vs cached — 10× on the read price, far less on the bill (TraceLab: 95.7% hits, prefix still 59.5% of cost); standard vs fast mode — 2× (both vendors), buying only writing speed.

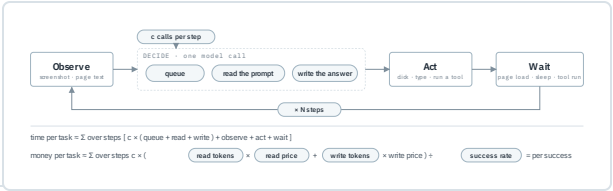
A5 · Each cause’s effect on time and on money



the complete table behind pages 16–17

CAUSE	EFFECT ON TIME	EFFECT ON MONEY	RELATIONSHIP
Many steps (N)	Roughly linear: one more step is one more call and one more wait (later steps somewhat slower, up to 3×)	Write tokens linear in N; read tokens quadratic, because every step re-reads the whole history	same source — double the steps: about 2× the time, about 4× the read bill (uncached)
Many calls per step (c)	linear	linear	same source
Reading a lot (history, screenshots, rich observations)	Reading time, longer every step	Read-token bill	same source — the same tokens cost time and money
Writing a lot (thinking)	Generation time, roughly 12 ms per token	Write-token bill at 5× the read price	same source , and the dearest time: writing is the slowest and the most expensive token
Cache hits	Saves reading time	Read price 0.1×	same source , but fragile: editing the history or switching to fast mode invalidates it
Queueing	slow	free	slow but not expensive ; escaping it means a fast or priority tier at 2×
Environment waits (page loads, fixed sleeps)	slow	No API cost; machine time by the hour — CPU sandboxes \$0.02–0.36 per hour, usually under 5% of the API bill	slow but not expensive
Tool tails (tests, builds)	slow	No API cost	slow but not expensive
Failures, idling, retries	slow	expensive	same source , and it turns “per attempt” into “per success”
Fast mode	Writing up to 2.5× faster (vendor-stated); reading unchanged	2× the price	money for time
Bigger models, more thinking	slower	dearer	money and time for accuracy , convex: marginal tokens per point 101K → 575K on one model; 6× / 9× / 9.6× tokens for a few points; more thinking not always more accurate
Smaller models	faster	cheaper	but if accuracy falls, steps and retries rise — possibly slower and dearer again (inference, no direct evidence)

A6 · The gaps in the evidence, and what each one blocks



the complete table behind page 18

GAP	WHAT EXISTS TODAY	CONSEQUENCE FOR WHAT CAN BE CLAIMED
Read / write time split and cache hit rate for a frontier API model	No measurement reports both; the only read / write split is on locally served 27–31B models (Yuan et al., 2026); screenshot-agent measurements do not report cache state	How much of “87–97% in the model” is reading, how much writing, and what remains once the cache is on — unknown
How much slower an agent is than a person	None of 17 benchmarks measures agent and human elapsed time on the same tasks; the closest is AXIS’s user study, in which a UI agent was 1.69× slower than manual work on easy tasks (small sample) (Lu et al., 2025)	“X times slower than a person” cannot go on a slide
Environment dollars	Benchmarks report instance types; AgentSysBench reports shares; products publish nothing. Computable from instance type × duration × list price: CPU environments usually 0.3–5% of the API bill	Every benchmark cost is an API bill; the omission is small for CPU environments, not for GPU sandboxes or long idle sessions
Whether OSWorld-Human’s “action” includes the 2-s sleep after each step	Not stated (xlang-ai, 2026); if it does, “actions under 2%” implies more than 100 s per step (calc.)	“Environment under 3.5%” may understate the waiting
The rebound from a smaller model	No direct evidence on how many extra steps and retries a drop in accuracy causes	The last row of the relationship table stays marked “inference”
How much slowness hurts accuracy	Only OSWorld 2.0 records the stale-state failure mode, without its share of failures (XLANG Lab, 2026)	“It exists” can be said; “how large” cannot